



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Triennale in Informatica

TESI DI LAUREA

Synaptic Coder: un Sistema LLM Multi-Agente per la Generazione Automatica di Codice Sorgente

RELATORE

Prof. Fabio Palomba

Dott. Antonio Della Porta

Università degli Studi di Salerno

CANDIDATO

Dorotea Serrelli

Matricola: 0512113740

Anno Accademico 2024-2025

Questa tesi è stata realizzata nel

sesa^{lab}
SOFTWARE ENGINEERING
SALERNO

"La cosa che è davvero, davvero sorprendente è che il modo in cui si programma un'intelligenza artificiale è come il modo in cui si programma una persona."

Jensen Huang

Abstract

Le straordinarie capacità dei Large Language Model (LLMs) nella generazione e comprensione del linguaggio naturale stanno guidando cambiamenti significativi in vari settori, inclusa l'ingegneria del software. Nonostante vari studi descrivano positivamente l'impatto di questi modelli di intelligenza artificiale (Artificial Intelligence - AI) nelle attività di sviluppo di un software, soprattutto in quella di generazione del codice sorgente, gli output che essi generano possono essere incoerenti in relazione ad una richiesta. Ciò ostacola la comprensione della risposta generata, oltre a sollevare nei professionisti grandi preoccupazioni sulla loro affidabilità. Per affrontare queste sfide, i ricercatori si sono focalizzati sul prompt engineering e sull'utilizzo di istanze multiple di LLM che, in più turni di discussione, costruiscono una risposta finale comune al problema. Sebbene noti framework LLM multi-agente come MetaGPT e AgentCoder coordinino agenti specializzati nella generazione di codice, entrambi seguono pipeline sequenziali, prive di meccanismi di dibattito esplicito, limitando la capacità del sistema di esplorare soluzioni alternative in modo competitivo e critico.

L'obiettivo di questa tesi è la creazione di un sistema LLM multi-agente in cui uno o più agenti autonomi collaborano, tramite dibattito, per assolvere compiti legati alla generazione di codice sorgente a partire da descrizioni in linguaggio naturale fornite dall'utente. Successivamente, sono state analizzate le prestazioni raggiunte dal sistema LLM multi-agente nella risoluzione del task precedentemente citato, e comparate con quelle raggiunte da un singolo LLM specializzato nella generazione di codice sorgente (o CodeLLM). Dall'analisi dei risultati, si è osservato che il sistema LLM multi-agente produce codice sorgente qualitativamente pari o superiore rispetto a quello generato da un singolo CodeLLM.

Questa ricerca dimostra, così, che non necessariamente l'aumento dei parametri di un LLM comporta sempre un miglioramento dell'efficacia e dell'accuratezza del modello nell'esecuzione di un compito.

Indice

Elenco delle Figure	iv
Elenco delle Tabelle	v
1 Introduzione	1
1.1 Contesto applicativo	1
1.2 Motivazioni ed obiettivi	3
1.3 Risultati	4
1.4 Struttura della tesi	4
2 Background e Stato dell'Arte	6
2.1 Large Language Model (LLM)	6
2.2 Code LLM	8

2.3	Collaborative Artificial Intelligence e LLM multi-agent systems . . .	9
3	Synaptic Coder : un sistema LLM multi-agente per la generazione automatica di codice sorgente	15
3.1	Domande di ricerca	15
3.2	Formulazione di agente e sistema multi-agente	17
3.3	La comunicazione nel sistema LLM multi-agente	19
3.3.1	Le caratteristiche generali di un canale di comunicazione . . .	19
3.3.2	Le caratteristiche della comunicazione adottate in Synaptic Coder	20
3.4	Architettura di Synaptic Coder	22
3.4.1	Gli agenti	22
3.4.2	L'insieme di obiettivi	22
3.4.3	L'input e l'output del sistema	23
3.4.4	Il flusso di lavoro in Synaptic Coder	32
4	Sperimentazione con Synaptic Coder	41
4.1	BigCodeBench: un benchmarking per la valutazione di Synaptic Coder	41
4.2	Processo di valutazione di Synaptic Coder	43
4.3	Risultati ottenuti	46
4.4	Analisi sulla qualità del codice sorgente generato da Synaptic Coder	48
4.5	Confronto delle prestazioni tra Synaptic Coder e il singolo CodeLLM	53
5	Conclusioni	56

5.1	Impatto	56
5.2	Sviluppi futuri	57
	Bibliografia	59

Elenco delle figure

3.1	Schema JSON di risposta del programmatore.	32
3.2	Schema JSON di risposta del valutatore.	33
4.1	Esempio di record del dataset BigCodeBench - 1	44
4.2	Esempio di record del dataset BigCodeBench - 2	45
4.3	Esempio di risposta del valutatore e di metriche di SonarQube per un codice sorgente	51

Elenco delle tabelle

4.1	Risultati del sistema LLM multi-agente per i primi 10 task di BigCode-Bench.	50
4.2	Risultati di un singolo CodeLLM per i primi 10 task di BigCodeBench.	54

CAPITOLO 1

Introduzione

1.1 Contesto applicativo

I recenti progressi sugli LLM hanno trasformato il campo dell'intelligenza artificiale generativa, grazie alla loro capacità di comprendere il linguaggio naturale a tal punto da guidare cambiamenti significativi in vari settori, inclusa l'ingegneria del software. Negli ultimi anni i ricercatori si sono focalizzati attivamente sui benefici derivanti dall'applicazione di questi modelli di AI nelle attività di ingegneria del software, documentando il loro impatto su produttività, qualità e risoluzione dei problemi durante lo sviluppo di un software. L'avvento di LLMs come ChatGPT, Code LLaMA, Code Gemma, DeepSeek-Coder, noti come Code LLM, ha trasformato profondamente il panorama dei compiti automatizzati legati al codice, inclusi il debugging ed il refactoring del codice sorgente.

Un'applicazione particolarmente interessante dei Code-LLM è la generazione di codice, un compito che implica la produzione di codice sorgente a partire da descri-

zioni in linguaggio naturale. Questo campo in forte espansione ha suscitato notevole interesse da parte sia dei ricercatori accademici che dei professionisti del settore, come evidenziato dallo sviluppo di strumenti come GitHub Copilot², CodeGeeX³ e Amazon CodeWhisperer⁴.

Nonostante vari studi descrivano positivamente l'impatto degli LLM nelle attività di sviluppo software, le prestazioni di tali modelli sono, talvolta, soggette al cosiddetto fenomeno delle "allucinazioni": gli output incoerenti ed, addirittura, irrilevanti in relazione ad una richiesta possono ostacolare la comprensione della risposta generata (es. il codice sorgente di un software), oltre a sollevare nei professionisti del settore grandi preoccupazioni sulla loro affidabilità.

Oltre al fenomeno delle "allucinazioni", i modelli presentano delle limitazioni intrinseche come natura autoregressiva (ad esempio, incapacità di un ragionamento lento e deliberativo) e vincoli imposti dalle leggi di scaling.

Per affrontare queste sfide, si è pensato di integrare gli LLM con strumenti esterni e meccanismi come la pianificazione per compiere azioni, risolvere problemi complessi, apprendere e interagire con ambienti esterni. Tale tipologia di LLM è conosciuta con il termine di agente.

In aggiunta, i ricercatori stanno sperimentando l'applicazione di molteplici agenti basati su LLM in un contesto collaborativo al fine di raggiungere un'intelligenza collettiva. Tali ricerche hanno portato allo sviluppo dei Sistemi Multi-Agente (Multi-Agent Systems, MAS) e dell'intelligenza artificiale collaborativa. Esse, infatti, mirano allo sviluppo di gruppi di agenti intelligenti in grado di coordinarsi, condividere conoscenze e risolvere problemi in modo collettivo.

Nel contesto della programmazione automatica, framework LLM multi-agente come MetaGPT e AgentCoder rappresentano approcci emergenti ed interessanti per migliorare la produttività nell'uso di LLM. Questi framework automatizzano lo sviluppo software simulando un team di lavoro (Project Manager, progettista dell'architettura del software, sviluppatore, tester), rendendo il processo più modulare e simile a un workflow reale. Tuttavia, gli agenti comunicano tramite prompt strutturati in un

flusso predefinito: gli agenti collaborano per migliorare il codice seguendo una pipeline statica, senza presentare soluzioni concorrenti per il task fornito, né confrontarle apertamente. Ciò limita la capacità del sistema di esplorare soluzioni alternative in modo competitivo e critico.

Si è visto, infatti, che applicando un approccio ispirato alla "society of minds" [1], si migliora significativamente il ragionamento matematico e la validità fattuale dei contenuti generati, riducendo le risposte errate e le allucinazioni a cui i modelli contemporanei sono soggetti. Tale approccio consiste nella definizione di più istanze di modelli linguistici che propongono e dibattono le loro risposte individuali ad un task di generazione di codice; si articola un processo di dibattito in cui i modelli discutono, in più round, le soluzioni generate, per, poi, arrivare ad una risposta finale comune.

Inoltre, studi recenti hanno evidenziato che l'adozione di sistemi multi-agente può superare le limitazioni di un singolo LLM, soprattutto quando si considerano modelli con iperparametri non voluminosi (es. la finestra di contesto), ottenendo vantaggi come una maggiore efficienza computazionale, una scalabilità superiore e prestazioni migliorate. Questi sistemi rappresenterebbero, quindi, un passo avanti nella progettazione di architetture LLM più efficienti e performanti.

1.2 Motivazioni ed obiettivi

La suddetta tesi si è incentrata sulla creazione di un sistema LLM multi-agente in cui uno o più agenti autonomi collaborano, tramite dibattito, per assolvere compiti legati alla generazione di codice sorgente a partire da descrizioni in linguaggio naturale fornite dall'utente.

Successivamente, si è valutato qualitativamente il codice sorgente generato dal sistema LLM multi-agente in termini di correttezza, manutenibilità, affidabilità e sicurezza. In aggiunta, sono state analizzate le prestazioni raggiunte dal sistema

LLM multi-agente nella risoluzione del task precedentemente citato, e comparate con quelle raggiunte da un singolo LLM specializzato nella generazione di codice sorgente.

La finalità di questa analisi è stata quella di confrontare le performance raggiunte del sistema multi-agente con quelle raggiunte da un singolo LLM, verificando se l'adozione del sistema LLM multi-agente può portare a una crescita logistica delle prestazioni, simile a quella osservata nei modelli neurali, ma con un miglioramento più rapido e meno costoso.

1.3 Risultati

È stato realizzato un sistema LLM multi-agente in cui uno o più agenti autonomi collaborano, tramite dibattito, per assolvere compiti legati alla generazione di codice sorgente a partire da descrizioni in linguaggio naturale fornite dall'utente.

L'analisi dei risultati raggiunti dal sistema ha dimostrato che, qualitativamente, il codice sorgente generato soddisfa i criteri di affidabilità, manutenibilità e correttezza; inoltre, risulta che il codice sorgente prodotto dal sistema è, in termini di correttezza, migliore rispetto a quello generato da un singolo agente specializzato nella generazione di codice sorgente.

1.4 Struttura della tesi

La tesi è strutturata nei seguenti capitoli:

- Background e Stato dell'Arte

In questo capitolo verranno forniti i concetti principali necessari alla comprensione dell'argomento trattato nella tesi, ognuno descritto nell'apposita sezione:

- Large Language Model (LLM);

- CodeLLM;
- Collaborative Artificial Intelligence e LLM multi-agent systems.

In ognuna delle sezioni sopra citate verranno esaminati gli articoli scientifici cruciali che esprimono i progressi nella costruzione di sistemi LLM multi-agenti e nell'uso degli LLM in attività legate alla generazione di codice sorgente.

- Metodo di Ricerca

In questo capitolo si andranno a dettagliare le domande di ricerca alle quali questa tesi si propone di rispondere, con la conseguente definizione di un sistema LLM multi-agente specializzato nel task di generazione di codice sorgente. Si partirà con la formulazione matematica di agente e sistema multi-agente, passando per la sua architettura, nonché il protocollo di comunicazione adottato al suo interno.

- Analisi dei risultati

In questo capitolo si descriverà come si è effettuata la valutazione del sistema LLM multi-agente, considerando come dati per la valutazione quelli memorizzati nel benchmarking BigCodeBench, e si analizzeranno i risultati prodotti. Successivamente, si osserverà come il sistema sviluppato ha risposto alle domande di ricerca ed alle sue prestazioni nella risoluzione del task rispetto ad un singolo LLM.

Background e Stato dell'Arte

2.1 Large Language Model (LLM)

I Large Language Models sono modelli di intelligenza artificiale che utilizzano il deep learning per comprendere, generare e manipolare il linguaggio naturale. Essi sono addestrati su immense quantità di dati per apprendere le complessità del linguaggio naturale e generare non solo testo, ma anche altri tipi di contenuti per l'esecuzione di un'ampia gamma di attività come traduzione di testi, analisi di immagini mediche, previsione di trend del mercato finanziario, fino ad attività legate alla gestione del ciclo di vita del software.

Modelli come GPT di OpenAI, LLaMA di Meta e la serie Gemini di Google si basano su reti neurali artificiali su larga scala, composte da miliardi, talvolta trilioni, di parametri. Lo sviluppo di un numero sempre più grande di iperparametri (es. parametri, dati, capacità di calcolo) e, più in generale, l'aumento della scalabilità degli LLM ha portato allo sviluppo di comportamenti o capacità che non si manifestano

nei modelli più piccoli (si tratta di abilità emergenti). Tali abilità permettono agli LLM di risolvere compiti per i quali non sono stati esplicitamente addestrati, come l'instruction following e il zero-shot learning, ovvero la capacità di affrontare nuovi problemi senza un'ulteriore fase di fine-tuning.

Il rilascio di modelli come GPT-3 negli ultimi anni ha reso queste capacità accessibili al pubblico, generando un'impennata nella ricerca accademica e industriale su come ottimizzare, scalare e rendere sicuri gli LLM per applicazioni nel mondo reale.

Nonostante le innovazioni promettenti, gli LLM presentano diverse criticità.

Le loro prestazioni tendono a degradarsi man mano che la conoscenza del mondo reale evolve, il che ha spinto l'attenzione verso metodi di allineamento dei modelli con informazioni aggiornate, senza dover riaddestrare i modelli completamente da zero. Inoltre, un modello può essere affetto da "allucinazioni": esso genera risposte che sembrano plausibili dal punto di vista linguistico ma sono, in realtà, false, inesatte o prive di fondamento. Ciò è dovuto, principalmente, al fatto che un LLM non "comprende" il testo fornito: prevede la parola o la frase più probabile data una sequenza di input.

Se nei dati di addestramento non c'è un'informazione chiara o coerente, il modello "indovina" basandosi su pattern simili.

Per affrontare queste sfide, i ricercatori hanno considerato diversi approcci. Ad esempio, in [2] si definisce l'auto-raffinamento iterativo (iterative self-refinement): è un processo che prevede la creazione di una bozza iniziale e il suo successivo perfezionamento sulla base del feedback fornito da se stessi. Tale processo può essere fornito dagli LLM senza un ulteriore addestramento, portando a risultati di qualità superiore su un'ampia gamma di compiti.

L'iterative self-refinement alterna due passaggi generativi: FEEDBACK e REFINE. Questi passaggi lavorano in tandem per generare risultati di alta qualità. Data una prima generazione prodotta da un modello M , essa viene fornita nuovamente al medesimo modello M per ottenere un feedback. Successivamente, il feedback viene reimmesso nello stesso modello per affinare la bozza precedentemente generata.

Questo processo viene ripetuto per un numero specificato di iterazioni o finché M non dichiara che non è necessario alcun ulteriore perfezionamento.

Si è visto che l'iterative self-refinement supera la generazione diretta da LLM forti come GPT-3.5 e GPT-4 (OpenAI, 2023) con un miglioramento assoluto dal 5% al 40%. Nei compiti di generazione di codice, tale algoritmo migliora la generazione iniziale fino al 13% assoluto quando applicato a modelli di codice potenti come Codex.

Un altro approccio considerato è quello di integrare gli LLM con strumenti esterni (es. tool per fare ricerche su Internet, lettura di file, interazioni con software di terze parti) e meccanismi come la pianificazione, per risolvere problemi complessi, apprendere e interagire con ambienti esterni. Un LLM capace di operare in queste modalità è definito agente.

Infine, si è delineata un'area di ricerca che esplora l'impiego di molteplici agenti basati su LLM in un contesto collaborativo: consentire a gruppi di agenti intelligenti di coordinarsi, condividere conoscenze e risolvere problemi in modo collettivo. La progettazione di sistemi di AI per lavorare insieme ad altri agenti intelligenti o con esseri umani è alla base dell'intelligenza artificiale collaborativa (Collaborative AI).

2.2 Code LLM

I Large Language Model per la generazione di codice (Code-LLM) mirano a generare codice sorgente a partire da descrizioni in linguaggio naturale, un processo noto anche come natural-language-to-code task (task di traduzione da linguaggio naturale a codice).

I Code-LLM seguono una pipeline di sviluppo articolata: inizia con la selezione e la sintesi dei dati di codice, seguita da un approccio strutturato di addestramento che comprende il pre-addestramento e il fine-tuning, l'apprendimento per rinforzo con diversi tipi di feedback, e l'utilizzo di tecniche avanzate di progettazione dei prompt (prompt engineering).

Tipicamente, le descrizioni in linguaggio naturale comprendono enunciati di problemi di programmazione e possono includere opzionalmente un contesto di programmazione (ad esempio, firme di funzioni, asserzioni, ecc...). Formalmente, si denotano con x le descrizioni in linguaggio naturale.

Dato x , l'uso di un LLM con parametri del modello θ per generare una soluzione di codice y può essere denotato come $P_\theta(y|x)$.

Grazie alla capacità di apprendimento contestuale, è possibile fornire ad un LLM una descrizione in linguaggio naturale x dettagliata di esempi come dimostrazioni per migliorare le prestazioni nella generazione del codice o per vincolare il formato della risposta.

Un set fisso di M esempi è denotato come $\{(x_i, y_i)\}_{i=1}^M$. Di conseguenza, una formulazione più generale dei Code-LLM con esempi few-shot (o zero-shot) può essere la seguente:

$$P_\theta(y|x) \Rightarrow P_\theta(y|\text{prompt}(x, \{(x_i, y_i)\}_{i=1}^k)), k = 0, 1, 2, \dots, M$$

dove $\text{prompt}(x, \{(x_i, y_i)\}_{i=1}^k)$ è una rappresentazione in stringa dell'input complessivo, e $\{(x_i, y_i)\}_{i=1}^k$ denota un set di k esempi selezionati casualmente da $\{(x_i, y_i)\}_{i=1}^M$. In particolare, per $k = 0$, si parla di generazione di codice zero-shot, equivalente alla generazione di codice tradizionale senza apprendimento contestuale.

2.3 Collaborative Artificial Intelligence e LLM multi-agent systems

La Collaborative AI coinvolge diversi ambiti, tra cui i sistemi multi-agente, l'interazione uomo-macchina, la teoria dei giochi e il trattamento del linguaggio naturale.

La collaborazione è l'elemento chiave che consente agli agenti di AI di interagire e lavorare insieme. Un meccanismo collaborativo semplice può consistere in due modelli che cooperano verso un obiettivo comune.

L'AI collaborativa segna una transizione dai modelli tradizionali ed isolati verso

approcci che pongono l'enfasi sulla cooperazione. Sono state proposte nuove metodologie per consentire agli agenti di interagire, negoziare, prendere decisioni, pianificare ed agire in maniera congiunta, spingendo in avanti le capacità dell'AI in contesti collettivi.

Un tema centrale dell'AI collaborativa è la ricerca sui sistemi multi-agente. Un sistema multi-agente (Multi-Agent System) è un sistema composto da molteplici agenti intelligenti che interagiscono tra loro. I componenti chiave di un sistema multi-agente sono:

- **Agenti:** Gli attori principali del sistema, dotati di ruoli, capacità, comportamenti e modelli di conoscenza. Capacità come apprendimento, pianificazione, ragionamento e presa di decisioni conferiscono intelligenza agli agenti e al sistema nel suo complesso.
- **Ambiente:** Il mondo nel quale gli agenti sono situati, da cui trarre informazioni e sul quale eseguire azioni. Gli ambienti possono essere simulati o reali, come fabbriche, strade, reti elettriche, ecc...
- **Interazioni:** Si intende la comunicazione tra gli agenti e comprende cooperazione, coordinamento, negoziazione ed altre dinamiche in base alle esigenze del sistema.
- **Organizzazione:** Gli agenti possono essere organizzati in modo gerarchico o auto-organizzarsi sulla base di comportamenti emergenti.

I sistemi multi-agente sono in grado di risolvere problemi che sarebbero difficili da affrontare per un singolo agente o un sistema monolitico. All'interno del sistema gli agenti risolvono compiti in modo collaborativo, offrendo al contempo maggiore flessibilità grazie alla loro capacità intrinseca di apprendere nuovi contesti e prendere decisioni in modo autonomo.

Le caratteristiche salienti di un sistema multi-agente sono:

- **Flessibilità e scalabilità:** è possibile aggiungere, rimuovere o modificare agenti affinché l'intero sistema possa adattarsi dinamicamente a contesti in evoluzione;

- Robustezza e affidabilità: la decentralizzazione del controllo consente al sistema di continuare a funzionare anche in presenza di componenti guasti, aumentando la tolleranza ai guasti e la resilienza.
- Auto-organizzazione e coordinamento: gli agenti possono autonomamente organizzarsi sulla base di regole di comportamento emergente (ripartizione del carico di lavoro, attuazione di decisioni coordinate, risoluzione di conflitti, ...).
- Operatività in tempo reale: il sistema è in grado di reagire immediatamente alle situazioni, senza necessità di supervisione umana, rendendo possibili applicazioni in scenari come il soccorso in caso di disastri e l'ottimizzazione del traffico.

L'efficienza dei sistemi multi-agente deriva dalla divisione del lavoro: un compito complesso viene suddiviso in sotto-compiti più semplici, ciascuno assegnato ad un agente distinto. I costi associati, come l'elaborazione e il consumo energetico, vengono ammortizzati tra i vari agenti, il che spesso si traduce in una soluzione meno onerosa rispetto ad un approccio centralizzato che affida l'intero problema complesso ad un'unica entità potente. Ogni agente può risolvere i compiti assegnati anche con un livello minimo di conoscenza predefinita, offrendo così un'elevata flessibilità.

Inoltre, la decentralizzazione del controllo conferisce al sistema un'elevata affidabilità: in caso di guasto di un agente, il compito può essere prontamente riassegnato ad altri agenti.

Recentemente si è dimostrato che un LLM può fungere da "mente" degli agenti individuali nei sistemi multi-agente. Nascono così gli agenti autonomi potenziati da LLM: sistemi dotati di sofisticate capacità di ragionamento, che sfruttano un LLM come motore computazionale centrale o controller. Ciò consente loro di operare all'interno di un ambiente condiviso, dove possono comunicare e interagire in modalità cooperativa, competitiva o negoziale, per inferire compiti e prendere decisioni in base allo stato del contesto.

L'architettura tipica di un tale agente comprende:

- un agente basato su LLM;
- un modulo di memoria: componente utile all'agente per memorizzare e recuperare le informazioni precedentemente generate;
- un componente di pianificazione: l'agente suddivide un compito complesso in sotto-obiettivi più piccoli e gestibili, oppure si impegna in processi di autocritica e auto-riflessione sulle azioni passate, al fine di apprendere dagli errori e migliorare le prestazioni future.
- un modulo di utilizzo degli strumenti: l'agente viene addestrato ad invocare funzioni esterne o APIs per svolgere il compito assegnato.

Nel campo della generazione automatica di codice, gli agenti autonomi basati su LLM hanno dimostrato una notevole competenza. Ne sono un esempio i seguenti framework:

- MetaGPT: framework innovativo di metaprogrammazione che integra l'efficienza dei flussi di lavoro umani nella collaborazione multi-agente basata su LLM.
- AgentCoder: un framework multi-agente composto da tre agenti specializzati, ciascuno con ruoli e capacità distinti. Questi ruoli includono:
 - un agente programmatore, responsabile della generazione del codice;
 - un agente progettista di test, incaricato della generazione dei casi di test unitari;
 - un agente esecutore di test, che esegue il codice e fornisce feedback.

Tuttavia, è da notare che anche nei contesti multi-agente sono evidenti le limitazioni dei sistemi LLM monolitici (natura autoregressiva, allucinazioni, ...), dove la complessità del coordinamento, della comunicazione e del processo decisionale è maggiore.

Frameworks come MetaGPT e AgentCoder introducono "tecniche di meta-programmazione, tra cui flussi di lavoro e processi strutturati nelle interazioni tra agenti, per scomporre e affrontare problemi complessi" [3], contribuendo così a mitigare tali problematiche. Inoltre, "meccanismi di ricerca del consenso, come quelli sperimentati nel progetto Consensus-LLM, dimostrano che gli LLM sono in grado di negoziare e allinearsi su obiettivi comuni in ambienti dinamici" [4].

In [1] si definisce un approccio ispirato al concetto "Society of Minds" in cui più agenti basati su LLM, dato un compito, propongono e discutono le proprie risposte ed i rispettivi processi di ragionamento in più turni di dibattito, al fine di giungere ad una risposta finale condivisa. I risultati ottenuti indicano che questo metodo "migliora in modo significativo il ragionamento matematico e la validità fattuale dei contenuti generati, riducendo le risposte fallaci e le allucinazioni a cui i modelli contemporanei sono soggetti".

Sebbene MetaGPT e AgentCoder coordinino agenti specializzati nella generazione di codice, entrambi seguono pipeline sequenziali prive di meccanismi di dibattito esplicito, limitando la capacità del sistema di esplorare soluzioni alternative in modo competitivo e critico.

Inoltre, risulta essere rilevante il legame tra lo sviluppo di sistemi LLM multi-agente e la crescita delle prestazioni.

Per un singolo modello LLM, le leggi di scala classiche (es. Scaling Laws for Neural Language Models – Kaplan et al., 2020) mostrano che l'aumento dei parametri, dei dati e del calcolo porta a prestazioni migliori. Tuttavia, a lungo andare, l'efficienza marginale decresce.

Per un sistema LLM multi-agente, le leggi di scala emergenti riguardano come le prestazioni migliorano o degradano al crescere del numero di agenti, delle interazioni tra di essi, o della complessità della loro comunicazione. Ad esempio, l'aggiunta di più agenti LLM piccoli (ossia con meno parametri) può aumentare la diversità, ridurre i costi e migliorare l'accuratezza, se coordinati in modo efficiente.

Un esempio è il framework Optima, nel quale l'uso di più agenti LLM con iperparametri poco voluminosi può portare a un miglioramento delle prestazioni fino a 2,8 volte, utilizzando meno del 10% dei token rispetto ai modelli tradizionali, grazie a una maggiore efficienza nella comunicazione e nell'elaborazione parallela. Questo approccio suggerisce che l'aumento del numero di agenti in un sistema multi-agente non solo è praticabile, ma può anche portare a una crescita logistica delle prestazioni, simile a quella osservata nei modelli neurali, ma con un miglioramento più rapido e meno costoso.

Questi lavori, in sintesi, dimostrano che la creazione di sistemi LLM multi-agente ha il potenziale per migliorare significativamente le capacità degli LLM ed aprire la strada ad ulteriori progressi nella generazione e comprensione del linguaggio.

Synaptic Coder : un sistema LLM multi-agente per la generazione automatica di codice sorgente

3.1 Domande di ricerca

Tenendo conto di ciò che è stato discusso nel capitolo "Background e Stato dell'Arte", questa tesi si propone di offrire una nuova soluzione al problema della generazione del codice sorgente mediante la creazione di un sistema LLM multi-agente. La metodologia di ricerca adottata per tale finalità risponde alle seguenti domande di ricerca:

Q RQ₁. *La qualità del codice sorgente generato dal sistema LLM multi-agente soddisfa i criteri di correttezza, sicurezza, manutenibilità, affidabilità?*

Q RQ₂. *Il codice sorgente generato dal sistema LLM multi-agente è qualitativamente migliore rispetto a quello generato da un singolo Code-LLM?*

Dunque, per la prima domanda di ricerca si osserverà qualitativamente il codice sorgente generato da Synaptic Coder in accordo ai seguenti criteri:

- **Correttezza:** un codice corretto esegue le funzionalità specificate e produce i risultati attesi in base ai requisiti forniti dall'utente, senza errori logici o di implementazione che portino a comportamenti inattesi o a output sbagliati.
- **Sicurezza:** un codice sicuro è progettato per prevenire vulnerabilità che potrebbero essere sfruttate da attaccanti malevoli.
- **Manutenibilità:** un codice manutenibile è facile da comprendere, testare, modificare ed estendere, sia per gli sviluppatori originali che per i nuovi membri del team.
- **Affidabilità:** un codice affidabile minimizza i guasti ed i malfunzionamenti, garantendo la continuità del servizio.

Per determinare tali criteri sul codice generato si farà uso di SonarQube, una piattaforma di analisi statica del codice che ispeziona il codice senza eseguirlo, identificando potenziali problemi e fornendo report dettagliati. Per farlo, utilizza una vasta collezione di regole (rules) che mappano le problematiche del codice ai criteri di qualità.

Una volta determinati tali criteri, si effettuerà il confronto qualitativo tra il codice sorgente generato da Synaptic Coder e quello generato da un singolo CodeLLM per la richiesta dell'utente, come riportato nella seconda domanda di ricerca.

Questo confronto assume un'importanza cruciale perché l'adozione di sistemi LLM multi-agente per questo task potrebbe presentare dei vantaggi in termini di efficienza computazionale (es. consumo di cicli di CPU, RAM) rispetto all'esecuzione di un solo Code-LLM di grandi dimensioni, vista la possibilità di distribuire il carico computazionale tra gli agenti.

3.2 Formulazione di agente e sistema multi-agente

In questa sezione si richiamano le definizioni matematiche di agente e sistema multi-agente riportate in [5], utili per descrivere l'architettura del sistema LLM multi-agente sviluppata.

Un agente $a = \{m, o, e, x, y\}$ è definito dalle seguenti caratteristiche:

- **Modello m :** il modello di AI $m = \{arch, mem, adp\}$, consistente nella sua architettura (*arch*), nel modulo di memoria (*mem*) e negli adapter opzionali (*adp*), ossia moduli intelligenti adattivi che permettono all'agente di incorporare conoscenza aggiuntiva da altri agenti. Nel caso degli agenti LLM, l'architettura *arch* è un modello linguistico e la memoria dell'agente *mem* è tipicamente il prompt di sistema r .
- **Obiettivo o :** guida le azioni dell'agente all'interno del sistema. Ad esempio, nei compiti di risposta a domande (question-answering), l'obiettivo è quello di generare una risposta che si avvicini il più possibile a quella corretta.
- **Ambiente e :** comprende lo stato o le condizioni in cui l'agente opera. Ad esempio, la finestra di contesto (context window) in un LLM è, solitamente, vincolata dal numero di token.
- **Input x :** la percezione in ingresso, come testo o dati da sensori. Negli LLM, x è codificato come una sequenza di token.
- **Output y :** l'azione o l'output corrispondente, definito dalla funzione $y = m(o, e, x)$, dove l'agente utilizza il suo modello m , il contesto e e l'obiettivo o per agire sull'input x .

Un sistema collaborativo multi-agente S , invece, è definito come segue:

- **Insieme di agenti $\mathcal{A} = \{a_i\}_{i=1}^n$:** insieme di n agenti basati su LLM, dove il numero di agenti può essere predefinito o adattato ai requisiti attuali del sistema.

- **Insieme di obiettivi** $\mathcal{O}_{collab}$: insieme collettivo di obiettivi che può essere partizionato in obiettivi unici per ciascun agente, garantendo l'allineamento con l'obiettivo generale del sistema.
- **Ambiente** $\mathcal{E}$: un ambiente condiviso dal quale gli agenti ricevono dati contestuali (es. database basati su vettori, interfacce di messaggistica comuni, ...).
- **Insieme di canali di comunicazione** $\mathcal{C} = \{c_j\}$: ogni canale di comunicazione facilita le interazioni tra gli agenti, permettendo lo scambio di informazioni basato su obiettivi, ambiente e input dati: $y_j = c_j(\{a_i(o_i, \mathcal{E}, x_i) | a_i, o_i, x_i \in c_j\})$. I canali si distinguono per i loro meccanismi, inclusi attori (agenti), tipi, strutture e strategie. Se due canali differiscono in questi aspetti, sono trattati come canali separati. Si assume che gli agenti abbiano un terreno comune, ossia che l'interfaccia può essere chiaramente compresa tra loro (ad esempio, tutti gli agenti usano l'inglese e sono attinenti all'argomento).
- **Input percepito dal sistema** x_{collab}
- **Output del sistema** y_{collab} : è modellato come

$$S(\mathcal{O}_{collab}, \mathcal{E}, x_{collab} | \mathcal{A}, \mathcal{C}) = \{c_j(\{a_i(o_i, \mathcal{E}, x_i) | a_i, o_i, x_i \in c_j\}) | c_j \in \mathcal{C}\}$$

dove ogni $c_j \in \mathcal{C}$ indica un canale di comunicazione che facilita le azioni collaborative degli agenti a_i , sulla base dei rispettivi input x_i , consentendo loro di lavorare insieme.

Attraverso questo flusso di lavoro strutturato, gli agenti in un sistema LLM multi-agente possono collaborare in modo adattivo, rispondendo ai requisiti del compito, apprendendo l'uno dall'altro e coordinando le azioni per raggiungere obiettivi condivisi.

3.3 La comunicazione nel sistema LLM multi-agente

3.3.1 Le caratteristiche generali di un canale di comunicazione

In un sistema LLM multi-agente è cruciale che gli agenti collaborino tra loro, condividendo un obiettivo comune o un insieme di obiettivi. In questo contesto, per "comportamento collaborativo" non si intende solo lo scambio di informazioni, ma anche il coordinamento e la gestione all'interno del sistema.

Ogni collaborazione è associata ad un canale di comunicazione e può essere di diversi tipi:

- **cooperazione:** gli agenti allineano i loro obiettivi individuali o_i con un obiettivo collettivo condiviso $\mathcal{O}_{collab} = \bigcup_{i=1}^n o_i$, lavorando insieme per raggiungere un risultato mutuamente vantaggioso.
- **competizione:** gli agenti danno priorità ai loro obiettivi individuali o_i , che potrebbero essere in opposizione agli obiettivi degli altri, introducendo un elemento di rivalità $\mathcal{O}_{collab} = \{o_i | o_i \neq o_j, \forall i \neq j\}$
- **coopetizione (coopetition) :** una fusione strategica di cooperazione e competizione, in cui gli agenti collaborano su determinate attività per raggiungere obiettivi condivisi, pur competendo contemporaneamente con altri. Applicata negli scenari di negoziazione, gli agenti assegnano valori differenti ai propri interessi, creando opportunità per compromessi e collaborazione.
- **ibrida :** combina le caratteristiche di ciascun tipo di collaborazione precedentemente trattato (es. competizione o cooperazione), sfruttando i punti di forza di ciascuno per migliorare le prestazioni complessive del sistema e la sua adattabilità.

Le strategie di collaborazione sono:

- **rule-based** : le interazioni tra gli agenti in $\mathcal{C}$ sono rigorosamente controllate da regole predefinite, garantendo che gli agenti coordinino le loro azioni secondo i vincoli globali del sistema sugli input accettabili x_{collab} .
- **role-based** : si sfruttano ruoli predefiniti distinti o divisioni del lavoro, in cui ogni agente $a_i \in \mathcal{A}$ opera su un obiettivo segmentato $o_i \subset \mathcal{O}_{collab}$ - solitamente basato sulla propria conoscenza del dominio - che supporta l'obiettivo complessivo del sistema.
- **model-based** : è applicato specialmente in ambienti in cui le incertezze nella percezione degli input possono influenzare le azioni degli agenti.

La struttura di un canale di comunicazione può essere di quattro categorie:

1. **centralizzata (o a stella)** : ogni agente è connesso ad un agente centrale, il quale ha l'obiettivo di gestire, controllare e coordinare le interazioni o collaborazioni tra gli agenti all'interno del sistema.
2. **decentralizzata** : si differenzia dai sistemi centralizzati per la distribuzione del controllo e la presa di decisioni tra gli agenti. Ogni agente opera sulla base di informazioni locali e, possibilmente, di una comunicazione limitata con altri agenti.
3. **stratificata** : la comunicazione è organizzata in livelli gerarchici, dove ogni livello ha agenti con funzioni distinte e che interagiscono principalmente all'interno del loro livello o con i livelli adiacenti.

3.3.2 Le caratteristiche della comunicazione adottate in Synaptic Coder

Tenendo conto delle caratteristiche di comunicazione descritte in precedenza, la comunicazione tra gli agenti all'interno del sistema LLM multi-agente sviluppato gode delle seguenti proprietà: cooperazione, rule-based e role-based, decentralizzata.

La cooperazione tra gli agenti è essenziale in compiti dove la risoluzione collaborativa dei problemi, la presa di decisioni collettive, e le competenze complementari contribuiscono a raggiungere obiettivi complessi che un singolo agente non potrebbe completare altrettanto efficacemente come la raccomandazione di prodotti, rispondere ad un questionario ed, appunto, la generazione di codice sorgente.

Inoltre, gli agenti possono essere utilizzati per concentrarsi su sotto-compiti specifici all'interno delle loro aree di competenza, migliorando l'efficienza e riducendo i tempi di completamento.

Per quanto riguarda la strategia di collaborazione, si è adottata una scelta ibrida tra rule-based e role-based.

I protocolli rule-based impongono una configurazione strutturata dei canali di collaborazione, in cui gli agenti agiscono sulla base di set di regole specifiche anziché di input probabilistici.

Inoltre, assegnando a ciascun agente una funzione specifica ed un insieme di compiti, le tecniche basate su ruoli conferiscono modularità alla struttura del sistema multi-agente e migliorano le prestazioni del sistema nel suo complesso.

La scelta di decentralizzare il controllo tra gli agenti è diffusa in vari settori, come la robotica (ad esempio, la robotica a sciame), i sistemi di rete (ad esempio, le reti di sensori) e l'intelligenza artificiale distribuita. È da notare, infatti, che in una struttura centralizzata, per quanto essa sia semplice da progettare, soffre del cosiddetto "single point-of-failure": se l'agente centrale collassa, allora diventa inutilizzabile l'intero sistema.

La struttura decentralizzata, dunque, risulta essere affidabile ed altamente scalabile; infatti, gli agenti possono operare in autonomia ed adattarsi ai cambiamenti del sistema.

3.4 Architettura di Synaptic Coder

In questa sezione si descriverà in dettaglio l'architettura del sistema LLM multi-agente che si è deciso di realizzare, discutendo anche delle problematiche affrontate nello sviluppo di tale soluzione.

3.4.1 Gli agenti

Il sistema è caratterizzato da n agenti specializzati nella generazione di codice sorgente, ossia basati su CodeLLM, dove n è il numero di agenti selezionati.

Tra gli n agenti si sono distribuiti i seguenti ruoli:

- programmatore o sviluppatore: fornita la descrizione di un problema in linguaggio naturale, egli genera un insieme di istruzioni, comandi e dichiarazioni scritte in un linguaggio di programmazione ad alto livello (il codice sorgente) che lo risolve;
- valutatore del codice sorgente (code reviewer): esamina il codice sorgente scritto dallo sviluppatore e genera
 - un esito numerico finale sulla qualità del codice sorgente;
 - un riscontro (feedback) che descrive le opportunità di miglioramento.

con l'obiettivo di identificare errori di compilazione, errori di esecuzione del codice, problemi legati alla correttezza, sicurezza, manutenibilità ed affidabilità.

In questa architettura ci saranno un solo valutatore e $n - 1$ programmatori.

3.4.2 L'insieme di obiettivi

Data la formulazione di un problema in linguaggio naturale dall'utente, il sistema LLM multi-agente ha l'obiettivo generale $\mathcal{O}_{collab}$ di fornire il codice sorgente cor-

rispondente alla richiesta, soddisfacendo qualitativamente i criteri di correttezza, sicurezza, manutenibilità ed affidabilità.

Poiché esiste una distinzione tra gli agenti con il ruolo di programmatore e code reviewer, è possibile definire due sotto-obiettivi che concorrono al raggiungimento di $\mathcal{O}_{collab}$:

- per lo sviluppatore: fornire un esempio di codice sorgente, corredato di documentazione in linguaggio naturale, che è soluzione del problema fornito dall'utente;
- per il valutatore: data la definizione del problema ed un esempio di codice sorgente, fornire una valutazione generale del codice sorgente ed un riscontro sui possibili miglioramenti del codice.

3.4.3 L'input e l'output del sistema

L'input percepito dal sistema x_{collab} e l'output generato y_{collab} è un insieme di token.

In particolare,

- x_{collab} sarà la formulazione del problema che si intende risolvere, espressa in linguaggio naturale;
- y_{collab} sarà il codice sorgente, formato da un blocco di imports, utile per il caricamento di librerie o moduli utilizzati nel codice, ed un blocco di istruzioni finito.

Un esempio di input del sistema può essere

Generate a function that implements the Fibonacci algorithm recursively.

e l'output corrispondente all'input sarà del seguente tipo:

```
1 def fibonacci(n):  
2     if n <= 0:  
3         return 0  
4     elif n == 1:  
5         return 1  
6     else:  
7         return fibonacci(n-1) + fibonacci(n-2)  
8
```

In un sistema LLM multi-agente di tipo cooperativo, più agenti LLM lavorano insieme per raggiungere un obiettivo comune. Tuttavia, anche in un contesto collaborativo, possono emergere svantaggi che compromettono l'efficacia del sistema:

- gli agenti possono interpretare l'obiettivo comune in modi leggermente diversi a causa del processo di addestramento e di fine-tuning che hanno ricevuto;
- se uno degli agenti produce un output errato, impreciso o fuorviante, gli altri potrebbero prendere per buone quelle informazioni, costruire ulteriori ragionamenti errati, e propagare l'errore a catena.

Tali criticità evidenziano la possibilità che gli agenti possano lavorare in direzioni leggermente divergenti, producendo risultati ridondanti, incoerenti o, addirittura, contrastanti. Ciò può compromettere il risultato finale e sprecare risorse computazionali.

Pertanto, si è avvertita la necessità di istruire ciascun agente sul compito da svolgere mediante prompt engineering.

Prompt engineering per istruire gli agenti ad un compito

Prompt engineering è una disciplina che consente di sviluppare e ottimizzare istruzioni (prompt) in modo strategico, al fine di ottenere risultati precisi ed utili dagli LLM per un'ampia varietà di applicazioni.

Una volta che l'utente ha fornito la descrizione del problema da risolvere, tale sequenza di token viene somministrata al sistema.

A questo punto, ciascun agente viene istruito sul compito mediante un prompt di ruolo (role prompting): si chiede all'agente di "assumere un ruolo specifico" per rispondere in modo più mirato e coerente, simulando il livello di conoscenza e il tono di un determinato tipo di esperto o personaggio.

Di seguito si riporta il prompt di ruolo adottato per un agente programmatore:

```

1  """You are an AI expert programmer that writes code or helps to review
2  code for bugs, based on the user request. Given a code generation task,
3  inserted in **CODE GENERATION TASK** section, provide a response
4  structured in the following JSON schema:
5
6  schema_complexity = {
7      "type": "object",
8      "properties": {
9          "documentation": {
10                 "type": "string",
11                 "description": "Description of
12     the problem and approach"
13             },
14             "imports": {
15                 "type": "string",
16                 "description": "Code block import statements"
17             },
18             "code": {
19                 "type": "string",
20                 "description": "Code block not including import statements"
21             },
22             "time_complexity": {
23                 "type": "string",
24                 "description": "Time complexity of the code block not i
25     ncluding import statements, expressed in Big-O notation"
26             }
27         },
28         "required": ["documentation", "imports", "code", "time_complexity"]
29     }
30
31  CODE GENERATION TASK EXAMPLE: Generate a Python function to add
32  two numbers.
33  JSON RESPONSE:
34  ...

```

```

35 {
36     "documentation": "The function 'add' takes two parameters
37     ('a' and 'b') and returns their sum ('a+b')."
38     "imports": "import sys",
39     "code": "def add(a, b): return a + b",
40     "time_complexity": "O(1)"
41
42 }
43 ...
44
45 CODE GENERATION TASK EXAMPLE: Generate a Python script about
46 binary search.
47 JSON RESPONSE:
48 ...
49 {
50     "documentation": "The binary search algorithm is an efficient way
51     to find an item from a sorted list. It works by repeatedly dividing
52     the search interval in half. If the value of the search key is less
53     than the middle item, the search continues in the lower half; if
54     it's greater, the search continues in the upper half. This process
55     continues until the value is found or the interval is empty.
56     The binary search function returns the index of the target if found,
57     otherwise -1.",
58     "imports": "import sys",
59     "code": "def binary_search(arr, target):
60         left, right = 0, len(arr) - 1
61         while left <= right:
62             mid = left + (right - left) // 2
63             if arr[mid] == target:
64                 return mid
65             elif arr[mid] < target:
66                 left = mid + 1
67             else:
68                 right = mid - 1
69         return -1"
70     "time_complexity": "O(log n)"
71 }
72
73 ...
74
75 CODE GENERATION TASK EXAMPLE: Generate a function to validate
76 a password. It checks if password given by the user has
77 a length of at least 8 and contains at least one number
78 and one letter.
79
80 JSON RESPONSE:
81 ...

```

```

82 {
83     "documentation": "The function 'check_password' checks
84     whether the input password has a length of at least 8
85     characters and contains at least one letter (alphabetic
86     character) and at least one number. It uses regular
87     expressions to ensure that the password contains both
88     alphabetic characters and digits. If the password meets
89     the requirements, the function returns True; otherwise,
90     it returns False.",
91     "imports": "import re",
92     "code": "def check_password(password):
93         # Check if the length is at least 8 characters
94         if len(password) < 8:
95             return False
96
97         # Check if the password contains at least
98         # one letter and one number
99         if not re.search(r'[a-zA-Z]', password):
100             # Check if there's at least one letter
101             return False
102         if not re.search(r'[0-9]', password):
103             # Check if there's at least one number
104             return False
105
106         return True"
107     "time_complexity": "O(n)"
108 }
109 ...
110
111 CODE GENERATION TASK
112 {user_prompt}
113
114 """

```

Si possono individuare nel role prompt per il programmatore le seguenti componenti:

1. una descrizione del ruolo che l'agente deve assumere e il suo obiettivo;
2. il formato della risposta che deve fornire al problema di generazione di codice sorgente;

3. una serie di esempi di problema fornito dall'utente, inerente alla generazione di codice sorgente, e della corrispondente risposta, nello schema JSON definito in precedenza;
4. una sezione, denotata *CODE GENERATION TASK* che tiene traccia del problema fornito in input dall'utente e al quale l'agente deve fornire una risposta.

Si riporta anche il role prompt adottato per l'agente valutatore:

```

1  """You are an AI expert source code evaluator.
2  We will provide you with the user input (the original coding prompt) and
3  an AI-generated code response. Your task is to analyze the quality of
4  the AI-generated source code according to the following criteria:
5
6  - Correctness: The code correctly implements the functionality
7  described in the user prompt and produces the expected output.
8  - Security: The code avoids common security vulnerabilities and
9  follows best practices for secure coding.
10 - Maintainability: The code is well-structured, readable, and
11 easy to understand and modify. Consider naming conventions,
12 modularity, comments, and adherence to coding standards.
13 - Reliability: The code handles edge cases and potential errors
14 gracefully, avoiding unexpected crashes or incorrect behavior.
15 - Compilation Errors: The number of syntax or compilation errors
16 that prevent the code from compiling.
17 - Execution Errors: The number of runtime or logical errors
18 that cause the code to fail during execution.
19
20 Follow these evaluation steps:
21 STEP 1: Carefully read the user input to fully understand the
22 coding task.
23 STEP 2: Assess the code response on Correctness, Security,
24 Maintainability, and Reliability, assigning a score from 0
25 (does not respect the criterion) to 100 (fully respects the
26 criterion) for each.
27 STEP 4: Verify if the code response has any Compilation
28 Errors and count them.
29 STEP 5: Verify if the code response has any Execution
30 Errors and count them.
31 STEP 6: Provide your evaluation strictly in the JSON
32 format described below.
33
34 The instruction for the coding task is provided in the
35 **User Input** section, and the AI-generated code is provided

```

```

36  in the **AI-generated Response** section.
37
38  Your response must be strictly in the following JSON schema:
39
40  ...
41  {
42      "type": "object",
43      "properties": {
44          "Correctness": {
45              "type": "integer",
46              "description": "The score from 0 to 100 about
47                  the correctness of the AI-generated code."
48          },
49          "Security": {
50              "type": "integer",
51              "description": "The score from 0 to 100 about
52                  the security of the AI-generated code."
53          },
54          "Maintainability": {
55              "type": "integer",
56              "description": "The score from 0 to 100 about
57                  the maintainability of the AI-generated code."
58          },
59          "Reliability": {
60              "type": "integer",
61              "description": "The score from 0 to 100 about
62                  the reliability of the AI-generated code."
63          },
64          "Compilation Errors": {
65              "type": "integer",
66              "description": "The number of compilation errors
67                  affected by the AI-generated code."
68          },
69          "Execution Errors": {
70              "type": "integer",
71              "description": "The number of execution errors
72                  affected by the AI-generated code."
73          },
74          "Explanation": {
75              "type": "string",
76              "description": "A detailed explanation of identified issues,
77                  including bugs, security vulnerabilities, execution errors,
78                  and compilation errors. Each issue must be clearly described,
79                  followed by one or more suggested fixes, formatted as a
80                  numbered list.
81                  This field can't be empty"
82          }

```

```

83     },
84     "required": [
85         "Correctness", "Security", "Maintainability", "Reliability",
86         "Compilation Errors", "Execution Errors", "Explanation"
87     ]
88 }
89 ...
90
91
92 EXAMPLE OUTPUT RESPONSE:
93 ...
94 {
95     "Correctness": 50,
96     "Security": 60,
97     "Maintainability": 45,
98     "Reliability": 20,
99     "Compilation Errors": 3,
100    "Execution Errors": 0
101    "Explanation": "The given code
102    ``` def calcola_media(lista):
103        somma = 0
104        for numero in lista:
105            somma += numero
106        media = somma / len(numero)
107        return media```
108    has the following issues and bugs:
109    - The line len(number) should be len(numbers). Since number is an
110    integer, calling len() on it causes a TypeError at runtime.
111    The logic is almost correct, but this bug prevents the function
112    from returning the correct result."
113 }
114 ...
115
116 # User Input
117 {user_prompt}
118
119 ## AI-generated Response
120 {ai_response}
121
122 """

```

Si possono individuare nel role prompt per il valutatore le seguenti componenti:

1. una descrizione del ruolo che l'agente deve assumere e il suo obiettivo;

2. una serie di istruzioni come guida per il valutatore nell'esecuzione del compito da svolgere;
3. il formato della risposta da fornire sottoforma di schema JSON;
4. un esempio di risposta nello schema JSON descritto nel punto precedente;
5. una sezione, denotata *USER INPUT* che tiene traccia del problema fornito in input dall'utente e al quale l'agente deve fornire una risposta;
6. una sezione, denotata *AI-generated Response*, che individua il codice del sistema LLM multi-agente che il valutatore deve esaminare.

Nella costruzione del prompt di ruolo sia del programmatore che del valutatore si è usata la tecnica di few-shot prompting per definire il formato della risposta che ogni agente deve generare; infatti, vengono forniti uno o più esempi nel prompt stesso, in modo che l'agente comprenda come deve rispondere.

Si è scelto di far generare come risposta da parte del programmatore uno schema in formato JSON, come riportato nella figura 3.1, contenente le seguenti proprietà:

1. **documentation**: spiegazione del codice generato e dell'approccio utilizzato per risolvere il problema;
2. **imports**: codice relativo all'importazione di librerie o funzioni presenti nel codice "code";
3. **code**: blocco finito di istruzioni, privo di imports;
4. **time_complexity**: la complessità temporale del codice sorgente "code", espressa in notazione Big-O.

Al compito di valutare qualitativamente il codice sorgente, l'agente valutatore genererà una risposta strutturata secondo lo schema JSON illustrato nella figura 3.2.

```

schema_complexity = {
  "type": "object",
  "properties": {
    "documentation": {
      "type": "string",
      "description": "Description of the problem and approach"
    },
    "imports": {
      "type": "string",
      "description": "Code block import statements"
    },
    "code": {
      "type": "string",
      "description": "Code block not including import statements"
    },
    "time_complexity": {
      "type": "string",
      "description": "Time complexity of the code block not including import statements, "
      "expressed in Big-O notation"
    }
  },
  "required": ["documentation", "imports", "code", "time_complexity"]
}

```

Figura 3.1: Schema JSON di risposta del programmatore.

3.4.4 Il flusso di lavoro in Synaptic Coder

Synaptic Coder esegue task di generazione automatica di codice sorgente rispettando il seguente workflow:

1. La descrizione del problema fornita dall'utente (user prompt) in input viene integrata nella costruzione del role prompt da fornire a ciascun agente programmatore.
2. Ogni programmatore istruito genera una propria risposta al problema dell'utente, fornendo tale soluzione nel formato descritto nella figura 3.1.
3. Da questo momento incomincia il dibattito: tra i programmatori inizia una discussione incentrata sulla scelta della migliore soluzione al problema, tra le $n - 1$ proposte avanzate, ossia il migliore codice sorgente. Il dibattito è strutturato in m turni, ciascuno caratterizzato dai seguenti avvenimenti:

```

evaluation_schema = {
  "type": "object",
  "properties": {
    "Correctness": {
      "type": "integer",
      "description": "The score from 0 to 100 about the correctness of the AI-generated code."
    },
    "Security": {
      "type": "integer",
      "description": "The score from 0 to 100 about the security of the AI-generated code."
    },
    "Maintainability": {
      "type": "integer",
      "description": "The score from 0 to 100 about the maintainability of the AI-generated code."
    },
    "Reliability": {
      "type": "integer",
      "description": "The score from 0 to 100 about the reliability of the AI-generated code."
    },
    "Compilation Errors": {
      "type": "integer",
      "description": "The number of compilation errors affected by the AI-generated code."
    },
    "Execution Errors": {
      "type": "integer",
      "description": "The number of execution errors affected by the AI-generated code."
    },
    "Explanation": {
      "type": "string",
      "description": "A detailed explanation for your assessment, including specific examples "
        "of issues and the error counts."
    }
  },
  "required": [
    "Correctness", "Security", "Maintainability", "Reliability",
    "Compilation Errors", "Execution Errors", "Explanation"
  ]
}

```

Figura 3.2: Schema JSON di risposta del valutatore.

- (a) Ogni programmatore sceglie il codice sorgente che presenta la migliore complessità temporale, espressa in notazione Big-O, ed è il più "leggibile". La leggibilità definisce la facilità da parte di un essere umano - in questo contesto, di altri programmatori - di comprendere e seguire il codice sorgente. Essa viene misurata mediante la metrica cognitive complexity.
- (b) Se tutti i programmatori concordano sulla stessa soluzione (**convergenza tra i programmatori**), allora il dibattito proclamerà il codice sorgente scelto come migliore soluzione al problema. Nel caso in cui esiste almeno un programmatore che promuove una soluzione diversa da quella degli altri programmatori (**divergenza tra i programmatori**), allora il dibattito prosegue.

Nel caso in cui si sono tenuti m turni di discussione e non si è giunti ad un accordo comune tra i programmatori, allora si procede con il voto di maggioranza (in inglese "**majority voting**"): il codice sorgente approvato dal dibattito sarà quello che è stato "votato" dalla maggioranza dei programmatori.

Nel caso in cui ci sia un pareggio, ovvero ci sono più codici sorgente che hanno avuto lo stesso massimo numero di voti, allora si dichiara fallito il dibattito.

4. Il codice sorgente approvato dal dibattito e l'utente prompt vengono integrati nella costruzione del role prompt del valutatore.
5. Una volta istruito il valutatore sul compito da svolgere, si comincia la valutazione:
 - (a) Il valutatore esamina il codice sorgente in termini di correttezza, sicurezza, manutenibilità, affidabilità, errori di compilazione ed esecuzione.
 - (b) Si fornisce per ciascun criterio di correttezza, sicurezza, manutenibilità ed affidabilità un valore da 0 (non soddisfatto) a 100 (pienamente soddisfatto), mentre per gli errori di compilazione ed esecuzione viene assegnato un intero che indica il numero di errori rilevati nel codice. In aggiunta, si genera una spiegazione per l'assegnazione dei valori, in accordo allo schema JSON in figura 3.2.
 - (c) Si calcola il punteggio totalizzato dal codice sorgente come media ponderata dei valori dati dal valutatore e i pesi assegnati per ciascun criterio.
6. A questo punto si osserva il punteggio finalizzato:
 - Il punteggio è maggiore o uguale a 85%: il sistema genera come output il codice sorgente come blocco di istruzioni composto da "imports" e "code" dello schema JSON in 3.1.
 - Il punteggio è minore di 85%: si ricomincia il dibattito tra i programmatori, stimolandoli a generare una nuova soluzione a partire dal codice precedentemente valutato e dal feedback del valutatore.
7. Se il numero di eventi dibattito-valutazione ripetuti supera il valore v , allora il sistema fornirà in output l'ultima soluzione valutata parzialmente corretta.

Nel punto 3 si è definito un dibattito tra gli $n - 1$ programmatori, costruito su m turni, per fornire il codice sorgente adatto a risolvere il problema dell'utente.

La scelta di impostare il dibattito tra gli agenti è legata allo studio [1], nel quale sono emersi diversi benefici relativi alla discussione tra agenti; infatti, si è visto che in un dibattito è meno probabile che gli agenti includano fatti falsi sui quali essi sono internamente incerti. Questo perché, man mano che il dibattito procede, gli agenti tendono a non essere d'accordo sui fatti incerti e ad ometterli dalla risposta. Infine, si è osservato che il dibattito non si limita ad amplificare una sola risposta corretta in un quorum di modelli: si sono verificati casi in cui tutti gli agenti inizialmente facevano predizioni incorrette e poi, con il progredire del dibattito, giungere alla risposta corretta.

Nel punto 3(a) ogni programmatore è tenuto a scegliere, tra $n - 1$ soluzioni, quella dotata dei valori migliori di complessità temporale e leggibilità. Nello specifico, si privilegia prima il valore della complessità temporale e, a parità di valori, quello della leggibilità.

La scelta di queste due caratteristiche come criteri discriminanti è basata sulle seguenti considerazioni:

- la complessità di tempo in notazione Big-O descrive come cresce il tempo (o il numero di operazioni) di un algoritmo al crescere della dimensione dell'input, indipendentemente dai dettagli implementativi ed hardware. Pertanto, per valori di complessità temporale molto grandi, le prestazioni del codice sorgente peggioreranno sempre di più.
- la complessità cognitiva permette di capire quanto sia "concettualmente complesso" seguire il flusso logico di un codice. Un codice con alta cognitive complexity è più difficile da leggere, testare, modificare e mantenere.

Dunque, visto l'impatto di questi due criteri su quelli che adotterà il valutatore per fornire un punteggio al codice sorgente, si vuole stimolare i programmatori a

scegliere soluzioni che, indirettamente, soddisfano il più possibile i criteri di sicurezza, manutenibilità, correttezza, affidabilità.

Il feedback generato dal valutatore verrà utilizzato dai programmatori per raffinare le proprie risposte date nel dibattito. L'adozione di questa metodologia è legata ai risultati dello studio [2].

Nella vita quotidiana, indirettamente l'essere umano adotta il raffinamento iterativo auto-guidato nella risoluzione di problemi: egli adotta un approccio iniziale, quasi rudimentale, alla risoluzione del problema per, poi, migliorarlo sulla base di un feedback auto-fornito.

Ad esempio, nel redigere un'email per richiedere un documento ad un collega, una persona potrebbe inizialmente scrivere una richiesta diretta come "Mandami il documento il prima possibile". Tuttavia, riflettendo sulla forma, il mittente potrebbe riconoscere la scortesia della formulazione e riscrivere l'email in "Ciao [collega], potresti inviarmi i dati appena ti è possibile?".

Analogamente, nella scrittura di codice sorgente, un programmatore potrebbe iniziare con un'implementazione "rapida e sporca", per poi, a seguito di una riflessione, rifattorizzare il codice in una soluzione più efficiente e leggibile.

Nel lavoro [2], gli autori hanno provato che gli LLM possono attuare un processo di raffinamento iterativo auto-guidato senza necessità di un addestramento aggiuntivo, generando output di qualità superiore in un'ampia gamma di compiti.

In virtù di queste considerazioni, nel flusso di lavoro del sistema LLM multi-agente è stato introdotto il raffinamento iterativo degli agenti programmatori, guidato dal feedback del valutatore.

Il problema della divergenza tra i programmatori

Osservando il flusso di lavoro del sistema LLM multi-agente, si è detto che se si riscontra una convergenza di opinioni dei programmatori sulla soluzione finale,

allora il codice sorgente scelto viene fornito al valutatore, decretando la fine del dibattito tra i programmatori.

Tuttavia, nonostante si siano definiti dei criteri oggettivi sui quali selezionare la migliore soluzione, per la natura regressiva e black-box di un generico LLM, si è riscontrato che, in caso di soluzioni proposte che presentano la stessa complessità di tempo e la stessa leggibilità, ogni agente programmatore sceglie come soluzione al problema dell'utente un determinato codice sorgente piuttosto che un altro, provocando il fenomeno della divergenza di opinioni.

Questo fenomeno è stato gestito mediante la definizione delle seguenti strategie:

1. **self-refinement (o raffinamento auto-guidato)**: ogni agente programmatore è stimolato a raffinare il codice sorgente che ha proposto nel precedente turno di dibattito, basandosi sui codici sorgente generati dagli altri programmatori.
2. **instant runoff voting (o voto alternativo)**: per ogni soluzione candidata da ciascun programmatore, se tale candidata è la scelta di più della metà dei votanti, viene dichiarata vincitrice. In caso contrario, viene rimossa dalla lista dei codici sorgente da scegliere la soluzione candidata preferita dal minor numero di votanti.
3. **mixed** : è una strategia che fonde il raffinamento auto-guidato con il voto alternativo. Fin quando non si verificano due turni di divergenza tra i programmatori nel dibattito, si stimola ciascun programmatore ad effettuare self-refinement. Superati i due turni di divergenza, si verifica se i codici sorgenti prodotti hanno stessa complessità di tempo e stessa cognitive complexity. In caso affermativo, si sceglie arbitrariamente uno dei frammenti di codice generati. In caso contrario, si procede con il voto alternativo.

Self-refinement

Nel lavoro [1] si è riscontrato che i modelli linguistici sono in grado di convergere su una risposta condivisa dopo diversi cicli di dibattito.

In particolare gli autori hanno visto che è possibile "controllare la durata del dibattito agendo, tramite richieste sviluppate con prompt, sulla fiducia che ciascun agente ha sul proprio output rispetto alle risposte generate dagli altri modelli".

Pertanto, si è sviluppato un instruction prompt che ha la finalità di stimolare ciascun agente a raffinare il proprio codice sorgente generato rispetto alle soluzioni proposte dagli altri agenti ad ogni turno di divergenza. Questo prompt conterrà:

- l'utente prompt;
- i codici sorgente proposti dagli altri agenti;
- il codice sorgente proposto dall'agente programmatore.

Instant runoff voting

Il problema delle decisioni multi-agente è ampiamente studiato nella teoria dei giochi cooperativi, in particolare nel campo della teoria della scelta sociale.

L'impostazione di base è la seguente. Dato un insieme di agenti $N = \{1, \dots, n\}$, detti votanti, essi vogliono esprimere una preferenza rispetto ad un insieme $\Omega = \{w_1, w_2, \dots\}$ di possibili risultati. In un'elezione politica, ogni elemento di Ω potrebbe rappresentare la vittoria di un diverso candidato.

Ogni votante avrà delle preferenze rispetto a Ω , che solitamente non sono espresse come utilità quantitative bensì come confronti qualitativi: si indica con $w >_i w'$ che il risultato w è valutato meglio del risultato w' da parte dell'agente i . In un'elezione con tre candidati, l'agente i può manifestare la seguente preferenza $w_2 >_i w_3 >_i w_1$.

Il problema fondamentale della teoria della scelta sociale è combinare queste preferenze, utilizzando una funzione di benessere sociale, per individuare un ordine di preferenza sociale: una classificazione dei candidati, dal più apprezzato al meno apprezzato. Si denoterà con $w >^* w'$ che w è classificato al di sopra di w' nell'ordine di preferenza sociale.

Tra le possibili procedure di voto (es. voto a maggioranza semplice per due candidati, il voto maggioritario, il voto per approvazione, ...), si è scelto il voto alternativo per risolvere la divergenza di opinioni nel dibattito.

In questa procedura di voto, i votanti classificano tutti i candidati e, se un candidato è la prima scelta di più della metà dei votanti, viene dichiarato vincitore. In caso contrario, il candidato indicato come primo dal minor numero di votanti viene eliminato, in modo che i votanti che avevano come prima scelta il candidato eliminato ora devono considerare un candidato diverso. A questo punto, la procedura si ripete fino a quando rimane un solo candidato con la maggioranza assoluta delle preferenze (a meno di un pareggio).

Nel contesto della realizzazione di Synaptic Coder, si è interessati solamente ad un risultato sociale: il risultato più apprezzato dal gruppo nel suo complesso. In tal caso, per ogni agente i non si chiederà di esprimere un ordine di preferenza per tutti i codici sorgente candidati, ma si chiederà di esprimere solamente una preferenza. Una funzione di scelta sociale, in tal caso, riceve come input la preferenza dichiarata da ciascun votante e restituisce un insieme di vincitori.

In questo modo, i programmatori sono orientati a scegliere il codice sorgente migliore tra un numero di scelte candidabili inferiore rispetto a quello presentato nel turno di dibattito precedente, cercando di favorire la convergenza.

Mixed

Nonostante i benefici descritti in precedenza a riguardo del self-refinement e dell'instant runoff voting, queste strategie presentano alcuni svantaggi.

Il self-refinement può consolidare errori, portare a loop di conferma, e consumare molte risorse, senza garanzia di miglioramento reale.

Il processo di iterazione e raffinamento, infatti, richiede risorse aggiuntive (tempo, calcolo). Il sistema deve elaborare più volte output, ricevere feedback, e applicare correzioni, aumentando la complessità operativa.

Poi, può accadere che, dopo alcune iterazioni, il sistema può stabilizzarsi su una so-

luzione che considera “buona abbastanza” e non trova ulteriori modi per migliorare.

Per quanto riguarda la strategia instant runoff voting, esso è un sistema di voto che migliora la rappresentatività rispetto al voto maggioritario semplice. Tuttavia, esso tende a premiare soluzioni con consenso trasversale, a scapito di soluzioni “specializzate” preferite solo da pochi agenti. Questo può ridurre la diversità di soluzioni considerate accettabili dal sistema.

Pertanto, si è pensato ad una strategia che permettesse ad un agente di migliorare la propria risposta al task dell’utente in funzione delle risposte degli altri agenti, senza compromettere in maniera netta le prestazioni del sistema o uniformare le risposte degli agenti programmatori.

In questa strategia si tiene traccia del numero di turni di dibattito in cui si verifica la divergenza tra i programmatori.

Fin quando non si verificano due turni di divergenza tra i programmatori nel dibattito, si stimola ciascun agente ad effettuare self-refinement. Superati i due turni di divergenza, si verifica se i codici sorgente prodotti hanno stessa complessità di tempo e stessa leggibilità. In caso affermativo, si sceglie arbitrariamente uno dei frammenti di codice generati. Si adotta tale criterio poiché tali agenti sono chiamati a fornire un codice sorgente che abbia la migliore complessità di tempo e la migliore leggibilità (cognitive complexity) senza entrare nel dettaglio di requisiti non funzionali come manutenibilità, affidabilità, ecc... .

In caso contrario, si procede con il voto alternativo.

Il sistema LLM multi-agente è stato creato come progetto in Python, reperibile su un repository GitHub mediante il seguente link: https://github.com/DoroteaSerrelli/BachelorThesis_LLM_multi-agent_system. Per l’utilizzo di LLMs in locale, si è fatto uso della libreria LMStudio.

Sperimentazione con Synaptic Coder

4.1 BigCodeBench: un benchmarking per la valutazione di Synaptic Coder

Per analizzare le prestazioni ed i risultati prodotti dal sistema LLM multi-agente sviluppato, si è somministrato il dataset BigCodeBench [6].

BigCodeBench è un benchmark di facile utilizzo per la generazione di codice con compiti di programmazione pratici e impegnativi¹. Il suo scopo è quello di valutare le reali capacità di programmazione degli LLM in un contesto più realistico. Il benchmark è progettato per compiti di generazione di codice a livello di funzione simili a quelli di HumanEval, ma con istruzioni molto più complesse e diverse chiamate di funzione.

¹GitHub - bigcode-project/bigcodebench: BigCodeBench: The Next <https://github.com/bigcode-project/bigcodebench/>

Il dataset² è caratterizzato da compiti di generazione di codice sorgente in Python (versione minima: 3.0) scritti in inglese. La struttura del dataset è la seguente:

- *task_id* (stringa): L'identificatore univoco del task.
- *complete_prompt* (stringa): Il prompt strutturato secondo PEP257³.
- *instruct_prompt* (stringa): Il prompt orientato al linguaggio naturale (scritto in inglese).
- *canonical_solution* (stringa): La soluzione canonica all'*instruct_prompt*, senza commenti.
- *code_prompt* (stringa): Il prompt contenente solo il codice.
- *test* (stringa): Il frammento di codice per il test, incapsulato in una classe *unittest.TestCase*.
- *entry_point* (stringa): Il punto di ingresso del frammento di codice, che è *task_func*.
- *doc_struct* (stringa[dizionario]): La docstring strutturata.
 - *description* (stringa): La descrizione principale del task di generazione del codice, espressa in linguaggio naturale (in inglese).
 - *note* (stringa): Note aggiuntive sul task in linguaggio naturale.
 - *reqs* (stringa, opzionale): I moduli che possono essere utilizzati nella soluzione del task.
 - *params* (stringa, opzionale): I parametri utilizzati nella soluzione del task.
 - *returns* (stringa, opzionale): I valori che devono essere restituiti nella soluzione del task.
 - *raises* (stringa, opzionale): Le eccezioni che devono essere sollevate nella soluzione del task.

²<https://huggingface.co/datasets/bigcode/bigcodebench>

³Python Enhancement Proposal 257: definisce le linee guida ufficiali per scrivere docstring (cioè stringhe di documentazione) in Python.

- *examples* (stringa, opzionale): Esempi interattivi in Python come suggerimenti per la soluzione del task.
- *libs* (stringa): Le librerie che possono essere utilizzate nella soluzione del task.

Tale benchmark soddisfa le seguenti tre aspettative riguardanti un buon benchmark di programmazione basato sull'esecuzione:

- Il benchmark dovrebbe essere facile da usare ed efficiente nella valutazione delle capacità fondamentali degli LLM. I benchmark a livello di repository o incentrati sugli agenti (es. SWE-bench) non sono adatti a questo scopo.
- Il benchmark dovrebbe essere pratico, coprendo diversi scenari di programmazione. I benchmark specifici per algoritmi (es. HumanEval e MBPP) non sono adeguati, così come i benchmark specifici per dominio (es. DS-1000).
- Il benchmark dovrebbe essere impegnativo, con tasks che richiedono solide capacità di ragionamento compositivo e di instruction-following da parte degli LLM. I benchmark con task semplici (es. ODEX) non sono adatti.

Pertanto, si è voluto adottare un benchmark che potesse valutare nel miglior modo possibile il sistema LLM multi-agente con task di programmazione impegnativi e pratici in un contesto aperto.

4.2 Processo di valutazione di Synaptic Coder

Per la valutazione di Synaptic Coder si è strutturato un processo con le seguenti fasi:

1. Assegnare lo stesso tipo di LLM sia agli $n - 1$ agenti programmatori che al valutatore.
2. Per ogni n°programmatori= 2, 3 e per ogni n° massimo di rounds di dibattito = 3, 4, 5

task_id	complete_prompt	instruct_prompt	canonical_solution
string	lengths	string	lengths
17	12.3k	232	78
BigCodeBench/0	import itertools from random import shuffle def task_func(numbers=list(range(1, 3))): """ Calculates the average of the...	Calculates the average of the sum of absolute differences between each pair of consecutive numbers for all permutations of a given list. Each...	permutations = list(itertools.permutations(numbers)) sum_diffs = 0 for perm in permutations: perm = list(perm) shuffle(perm) diffs = ..
BigCodeBench/1	import collections import random import string def task_func(length=100): """ Generate a random string of the specified length composed of uppercase...	Generate a random string of the specified length composed of uppercase and lowercase letters, and then count the occurrence of each character in thi...	if length < 0: raise ValueError random_string = ''.join(random.choices(string.ascii_uppercase + ..
BigCodeBench/2	import random import statistics def task_func(LETTERS): """ Create a dictionary in which keys are random letters and values are lists of random...	Create a dictionary in which keys are random letters and values are lists of random integers. The dictionary is then sorted by the mean of the...	random_dict = {} for _ in range(100) for _ in range(random.randint(1, 10)) for k in LETTERS: sorted_dict += ..
BigCodeBench/3	import random import numpy as np def task_func(LETTERS): """ Create a dictionary where keys are specified letters and values are lists of random...	Create a dictionary where keys are specified letters and values are lists of random integers. Then calculate the mean of these integers for each ke...	random_dict = {} for _ in range(100) for _ in range(random.randint(1, 10)) for k in LETTERS: sorted_dict += ..
BigCodeBench/4	from collections import Counter import itertools def task_func(d): """ Count the occurrence of each integer in the values of the input...	Count the occurrence of each integer in the values of the input dictionary, where each value is a list of integers, and return a...	count_dict = Counter(itertools.chain.from_iterable(d.values())) return dict(count_dict)
BigCodeBench/5	import random import math def task_func(LETTERS=list('a')) for i in range(97, 123)): """ Create a dictionary where keys are letters from a predefined...	Create a dictionary where keys are letters from a predefined list LETTERS and values are lists of random integers. Then, calculate the population...	random_dict = {} for _ in range(100) for _ in range(random.randint(1, 10)) for k in LETTERS: sorted_dict += ..
BigCodeBench/6	import os import re def task_func(pattern, log_dir="/var/log/"): """ Find the latest log file in a specified directory that matches a given regex...	Find the latest log file in a specified directory that matches a given regex pattern. This function searches through all files in the specified...	log_files = [] for f in os.listdir(log_dir): if re.match(pattern, f): log_files = sorted(log_files, key=lambda f: ..
BigCodeBench/7	import csv import collections import operator def task_func(csv_file_path): """ Find the best-selling product from a given...	Find the best-selling product from a given CSV file with sales data. This function parses a CSV file assumed to have a header followed by rows...	with open(csv_file_path, 'r') as f: reader = csv.reader(f) next(reader) # Skip the header row sales_data = ..
BigCodeBench/8	from collections import Counter import itertools from random import randint def task_func(T1, RANGE=100): """ Convert elements in 'T1' to...	Convert elements in 'T1' to integers and create a list of random integers where the number of integers is determined by the sum of the integers in...	int_list = [list(map(int, x)) for x in T1] flattened_list = list(itertools.chain(*int_list)) total_nums = sum(flattened_list)...
BigCodeBench/9	import pandas as pd import matplotlib.pyplot as plt import seaborn as sns def task_func(list_of_pairs): """ Create a Pandas DataFrame from a list o...	Create a Pandas DataFrame from a list of pairs and visualize the data using a bar chart. The title of the barplot should be set to 'Category'...	df = pd.DataFrame(list_of_pairs, columns=['Category', 'Value']) plt.figure(figsize=(10, 8)) sns.barplot(x='Category', y='Value'...
BigCodeBench/10	import numpy as np import itertools import random def task_func(T1, max_value=100): """ Convert elements in 'T1' to integers and...	Convert elements in 'T1' to integers and create a list of random integers. The size of the list is the sum of the integers in 'T1'. Calculate and...	if len(T1) <= 0: raise statistics.StatisticsError int_list = [list(map(int, x)) for x in T1] flattened_list = list(itertools.chain(*int_list)) total_nums = sum(flattened_list)...
BigCodeBench/11	import numpy as np import itertools import random def task_func(T1, max_value=100): """ Converts elements in 'T1' to a tuple of tuples...	Converts elements in 'T1' to a tuple of tuples containing string representations of integers, to integers and creates a list of random...	int_list = [list(map(int, x)) for x in T1] flattened_list = list(itertools.chain(*int_list)) total_nums = sum(flattened_list)...
BigCodeBench/12	import subprocess import os import json from datetime import datetime def task_func(script_name='backup.sh', log_file="/home/user/backup_log.json"):	Runs the provided backup shell script and logs the start time, end time, and exit status in a specified JSON log file. The function should raise...	log_data = {} if not os.path.isfile(script_name): raise FileNotFoundError(f"Script {script_name} does not exist.")
BigCodeBench/13	import subprocess import ftplib import os def task_func(ftp_server='ftp.duke.com', ftp_user='dlpuser'):	Download all files from a specific directory on an FTP server using wget in a subprocess. Args: ftp_server (str): The FTP server address. Default...	# Attempt to connect to the FTP server try: ftp_obj = ftplib.FTP(ftp_server) except Exception as e: raise
BigCodeBench/14	import configparser import os import shutil def task_func(config_file_path, archive_dir="/home/user/archive"): """...	Archive a specified project directory into a ZIP file based on the configuration specified in a config file. This function reads a...	config = configparser.ConfigParser() config.read(config_file_path) project_dir = config.get('Project', 'directory') if not ..
BigCodeBench/15	import subprocess import csv import os def task_func(commands_file_path, output_dir_path): """ Execute a list of shell commands read from a CSV...	Execute a list of shell commands read from a CSV file and save the outputs in separate files. Each command's output is written to a unique file in...	# Check if commands file path exists if not os.path.exists(commands_file_path): raise FileNotFoundError(f"File ..

Figura 4.1: Esempio di record del dataset BigCodeBench - 1

(a) Somministrare il dataset BigCodeBench al sistema LLM multi-agente, osservando i risultati ottenuti per strategia di gestione della divergenza adottata (self-refinement, instant runoff voting, mixed).

Pertanto, si è valutato il sistema tenendo conto delle tre differenti metodologie di gestione della divergenza durante il dibattito.

Inizialmente, si è richiesto agli agenti di generare, in maniera indipendente, la soluzione al problema fornito dall'utente e, in seguito, effettuare il self-refinement sulle risposte prodotte in caso di divergenza.

Successivamente, si è esaminata la generazione di risposte mediante l'impiego di agenti multipli con il voto alternativo per gestire la divergenza di opinioni.

Infine, si è eseguito il sistema con la strategia mixed.

I modelli LLM utilizzati nel sistema LLM multi-agente sono:

- qwen2.5-coder-3b-instruct: dotato di 3,09 miliardi di parametri e context window di 32.768 tokens;

caso cui si vogliono assegnare LLM diversi tra $n - 1$ agenti programmatori e l'agente valutatore.

4.3 Risultati ottenuti

Si sono sviluppati due file CSV per tenere traccia dei risultati sia del sistema LLM multi-agente che del singolo CodeLLM scelto per i compiti di generazione del codice sorgente. I file CSV, denotati `multi-agent_csv_results.csv` e `single-agent_csv_results.csv`, sono presenti nella cartella Code del progetto GitHub.

Nei file `multi-agent_csv_results.csv` si terrà traccia, per ogni task somministrato a Synaptic Coder, delle seguenti informazioni:

1. *task_id* : l'identificativo del task di BigCodeBench che è stato somministrato a Synaptic Coder;
2. *instruction_prompt* : è la descrizione in linguaggio naturale del task di BigCodeBench con identificativo *task_id*;
3. *canonical_solution* : è la soluzione del task proposta dal benchmark;
4. *code_multiagent_system*: è il codice sorgente generato da Synaptic Coder per il task *task_id*;
5. *documentation* : è la documentazione generata dal sistema per il codice *code_multiagent_system* generato;
6. *time_complexity*: la complessità temporale di *code_multiagent_debate* espressa in notazione Big-O;
7. *evaluation_feedback* : è la valutazione su *code_multiagent_system*, generata dal valutatore, espressa nel formato JSON 3.2;
8. *number_agents* : numero di agenti programmatori coinvolti nel dibattito;

9. *metrics_sonarqube*: tiene traccia dei valori di metriche di SonarQube usate per l'analisi qualitativa di *code_multiagent_system*;
10. *type_models*: il tipo di LLM adottato per gli agenti programmatori e per l'agente valutatore;
11. *max_rounds*: massimo numero di round di dibattito ammessi;
12. *time*: tempo impiegato da Synaptic Coder per eseguire il task *task_id*;
13. *debate_strategy*: la strategia utilizzata per gestire la divergenza di opinioni nel dibattito (0 = self-refinement, 1 = instant runoff voting, 2 = mixed);
14. *tests_success* : il numero di test, proposti da BigCodeBench per il task *task_id*, che sono stati soddisfatti da *code_multiagent_system*;
15. *tests_fails* : il numero di test, proposti da BigCodeBench per il task *task_id*, che non sono stati soddisfatti da *code_multiagent_system*.

Tali colonne sono presenti anche nel file `single-agent_csv_results.csv`.

In ciascun problema somministrato, si è osservato che l'impiego di agenti multipli per generare soluzioni equipara o migliora le prestazioni rispetto al singolo CodeLLM.

Contestualmente, si è anche rilevato che il processo di iterative self-refinement applicato al singolo CodeLLM produce generalmente un modesto incremento delle prestazioni. Il dibattito multi-agente, inteso come combinazione di self-refinement e generazione multi-agente, offre un incremento sostanziale nella capacità di ragionamento per ciascuno dei compiti.

È interessante notare che si sono riscontrati casi in cui uno o più agenti, inizialmente, non fornivano una risposta al task o generavano codice con qualche errore di compilazione e/o di esecuzione per, poi, giungere ad un codice sorgente comunemente approvato, privo di errori di compilazione e di esecuzione. Dunque, ciò conferma che il dibattito non porta ad amplificare un'unica risposta corretta o a perpetuare una risposta errata.

4.4 Analisi sulla qualità del codice sorgente generato da Synaptic Coder

Richiamando il flusso di lavoro di Synaptic Coder discusso nel capitolo "Architettura del sistema LLM multi-agente", se il codice sorgente generato ha un punteggio finale superiore a 85%, allora esso viene fornito come output del sistema.

Il punteggio finale viene calcolato come media pesata del valore dei criteri che l'agente valutatore assegna a ciascuno dei seguenti criteri: correttezza, sicurezza, affidabilità, manutenibilità. I pesi assegnati a ciascun criterio sono stati determinati in fase di sperimentazione:

- correttezza (c_weight): 35%;
- affidabilità (r_weight): 25%;
- sicurezza (s_weight): 20%;
- manutenibilità (m_weight): 20%.

Di seguito si mostra la formula per il calcolo della media ponderata, denotata con b_s (abbreviazione di *base_score*):

$$b_s = \frac{\text{correttezza} * (\text{c_weight}) + \text{sicurezza} * (\text{s_weight}) + \text{manutenibilit\`a} * (\text{m\_weight}) + \text{affidabilit\`a} * (\text{r_weight})}{100}$$

Inoltre, per garantire che il codice sorgente fornito in output da Synaptic Coder non contenesse errori di compilazione o errori di esecuzione, si è deciso di applicare una penalità per la generazione di risposte contenenti tali errori. Dal punteggio *base_score* ottenuto, si va a sottrarre il numero di errori di compilazione e di esecuzione ottenuti per i rispettivi pesi.

Anche in questo caso i pesi assegnati a tali criteri sono stati determinati sperimentalmente:

- errore di compilazione (compile_weight): 30%;

- errore di esecuzione (*exec_weight*): 15%.

Dunque, la penalità, denotata con p_s (abbreviazione di *penalty_score*), viene calcolata come

$$p_s = \#errori_compilazione * (compile_weight) + \#errori_esecuzione * (exec_weight)$$

ottenendo come punteggio finale per il codice sorgente:

$$final_score = base_score - penalty_score$$

Nel caso in cui $penalty_score > base_score$ il codice sorgente avrà $final_score = 0$.

Sottoponendo Synaptic Coder ai primi 10 problemi di generazione di codice sorgente presente nel dataset BigCodeBench, si sono ottenuti i risultati, in termini di correttezza, descritti nella tabella 4.1. Nella suddetta tabella è riportata la correttezza del codice prodotto per ciascuna strategia di gestione di divergenza nel dibattito tra due agenti, basati su *qwen – 2.5 – coder – 3b – instruct*, in 4 turni di dibattito consentiti. La correttezza del codice reale viene calcolata, in percentuale, come numero di test superati sul totale di test somministrati. È da precisare che è stato sottoposto al sistema LLM multi-agente lo stesso task per ben 5 volte.

Si è osservato che, indipendentemente dal numero massimo di turni di dibattito ($m = 3, 4, 5$), gli agenti tendevano a convergere verso un'unica soluzione già ai primi due turni di dibattito. Le soluzioni fornite dal dibattito ai primi turni risultavano avere, in alcuni casi, errori di compilazione e/o di esecuzione per cui era necessario innescare nuovamente il dibattito. Aumentando il numero di turni di dibattito m , dunque, si è ottenuto un miglioramento nel codice generato in termini di correttezza.

Per quanto riguarda il numero di agenti, si è provato ad aumentare il numero di agenti da 2 a 3 mantenendo fisso il numero massimo di rounds di dibattito a 3. Si è notato che, nel caso di 3 agenti, l'output fornito dal sistema LLM multi-agente aveva un punteggio finale compreso nel range 80%-85% per la presenza di qualche errore di

Tabella 4.1: Risultati del sistema LLM multi-agente per i primi 10 task di BigCodeBench.

Task ID	%Self-refinement	%Instant runoff voting	%Mixed
0	32%	14%	26%
1	100%	80%	80%
2	64%	92%	84%
3	100%	100%	100%
4	100%	80%	100%
5	92%	92%	84%
6	52%	52%	44%
7	100%	83%	97%
8	15%	20%	15%
9	100%	80%	100%

compilazione individuato dal valutatore. Aumentando, dunque, il numero massimo di turni di dibattito, si è ottenuto, in generale, un miglioramento del punteggio finale del codice prodotto.

È da tenere conto che i risultati di dibattito sono riportati utilizzando molteplici istanze dello stesso LLM.

Per ogni task di generazione del codice sorgente si è tenuta traccia della risposta del valutatore nel file `multi-agent_csv_results.csv`, espressa nel formato 3.2, e dei valori reali dei criteri di sicurezza, manutenibilità ed affidabilità, calcolati mediante SonarQube, e la correttezza.

Un esempio di confronto tra la risposta del valutatore e le metriche di SonarQube è mostrato nella figura 4.3.

Un bug in SonarQube è definito come un errore che ha un impatto certo e immediato sul comportamento del codice. Questi bug sono violazioni di regole che indicano un

evaluation_feedback ▾	number_agents ▾	metrics_sonarqube ▾
{ "Compilation Errors": 0, "Correctness": 100, "Execution Errors": 0, "Explanation": "", "Maintainability": 85, "Reliability": 100, "Security": 75 }	2	coverage: 0.0, bugs: 2, reliability_rating: 1.0, code_smells: 1, duplicated_lines_density: 0.0, cognitive_complexity: 3, security_rating: 5.0, ncloc: 15, vulnerabilities: 1, sqale_rating: 1.0

Figura 4.3: Esempio di risposta del valutatore e di metriche di SonarQube per un codice sorgente

difetto di programmazione che probabilmente causerà un risultato errato o un crash.

Questa metrica, dunque, fornisce, a grandi linee, indicazioni riguardo la possibile presenza di errori di compilazione nel codice. Combinando questo attributo con il numero di test, proposti da BigCodeBench per ciascun task, che il codice sorgente soddisfa, si può quantificare la correttezza del codice prodotto dal sistema.

Per quanto riguarda la sicurezza, la metrica *security_rating* misura la sicurezza del codice in base a

- vulnerabilità (vulnerabilities): problemi di sicurezza che rappresentano una minaccia diretta, come iniezioni SQL, XSS, o credenziali hardcoded.
- Security Hotspots: sezioni di codice sensibili alla sicurezza che richiedono una revisione manuale da parte dello sviluppatore

Questa metrica assume valori da 1 (nessuna vulnerabilità) a 5 (vulnerabilità critiche) oppure in una scala da A (nessuna vulnerabilità) ad E (vulnerabilità critiche).

SonarQube fornisce la metrica *sqale_rating* per avere informazioni circa la manutenibilità del codice. Questa metrica si avvale dell'analisi dei code smell nel codice per valutare la manutenibilità e la leggibilità del codice. Anche questa metrica assume valori da 1 (facile da mantenere) a 5 (criticità nella manutenibilità) oppure una scala di valori da A (facile da mantenere) a E (difficile da mantenere).

Analogamente alla manutenibilità, SonarQube assegna un "Reliability Rating" (Valutazione di Affidabilità) basato sul numero e sulla gravità dei bug rilevati. L'affidabilità è misurata principalmente attraverso l'identificazione di "Bugs" (come menzionato per la correttezza) e la capacità del codice di gestire gli errori. Analogamente alle metriche precedenti, *reliability_rating* assume valori da 1 (altamente affidabile) a 5 (affidabilità molto bassa) oppure una scala di valori da A (altamente affidabile) a E (affidabilità molto bassa).

Confrontando i valori delle metriche di SonarQube e i valori di correttezza, manutenibilità, affidabilità e sicurezza stimati dal valutatore per ciascun task, si è osservato che il valutatore

- sovrastima o sottostima la correttezza del codice tra 5% e 20%;
- non fornisce sempre un valore per la sicurezza concorde con *security_rating*: è possibile che questa discordanza sia legata al fatto che il valutatore non effettua analisi approfondite di problemi di sicurezza (es. iniezioni SQL, XSS, o credenziali hardcoded) come SonarQube;
- fornisce un valore di manutenibilità e di affidabilità circa uguali alle metriche *sqale_rating* e *reliability_rating*.

Pertanto, si evince che, in media, la qualità del codice generato dal sistema LLM multi-agente soddisfa i criteri di manutenibilità, affidabilità e correttezza, mentre non sempre è soddisfatto il criterio di sicurezza.

4.5 Confronto delle prestazioni tra Synaptic Coder e il singolo CodeLLM

In questa sezione si discuteranno i risultati qualitativi del codice sorgente generato da un singolo CodeLLM e si porranno a confronto con quelli relativi al codice sorgente prodotto da Synaptic Coder.

Il processo di valutazione del codice generato dal CodeLLM è analogo a quello applicato al sistema LLM multi-agente; infatti, si è adottato il benchmark BigCodeBench per comparare in modo agevole i risultati prodotti dal CodeLLM con quello di Synaptic Coder.

I risultati sono stati analizzati tenendo conto del numero di raffinamenti possibili sulla risposta iniziale del modello e sulla tipologia di CodeLLM scelta.

Sottoponendo il singolo CodeLLM ai primi 10 problemi di generazione di codice sorgente presente nel dataset BigCodeBench, si sono ottenuti i risultati, in termini di correttezza, descritti nella tabella 4.2. Nella suddetta tabella si è calcolata la correttezza del codice prodotto tenendo conto di 4 iterazioni di self-refinement. La correttezza del codice reale viene calcolata, in percentuale, come numero di test superati sul totale di test somministrati. Analogamente al sistema LLM multi-agente, si è somministrato lo stesso task al singolo CodeLLM per ben 5 volte.

Confrontando i risultati ottenuti in 4.1 e in 4.2 si giunge alla conclusione che, in media, per un task di generazione del codice sorgente, Synaptic Coder produce un risultato che è migliore di quello generato da un singolo CodeLLM in termini di 19,8%.

Osservando in dettaglio i risultati per ogni strategia di gestione di divergenza del dibattito, in termini di correttezza, risulta che

- per la strategia self-refinement si ha un miglioramento del codice rispetto a quello prodotto dal singolo CodeLLM, in media, di 23%;

Tabella 4.2: Risultati di un singolo CodeLLM per i primi 10 task di BigCodeBench.

Task ID	Tipo di modello	% Correttezza
0	codellama-13b-instruct	20%
1	codellama-13b-instruct	60%
2	codellama-13b-instruct	8%
3	deepseek-coder-v2-lite-instruct	100%
4	deepseek-coder-v2-lite-instruct	100%
5	deepseek-coder-v2-lite-instruct	92%
6	deepseek-coder-v2-lite-instruct	48%
7	starcoder2-7b	43%
8	starcoder2-7b	14%
9	starcoder2-7b	40%

- per la strategia instance runoff voting, si ha un miglioramento del codice rispetto a quello prodotto dal singolo CodeLLM, in media, di 16,8%;
- per la strategia mixed, il codice prodotto è migliore del 20,5% rispetto a quello prodotto dal singolo CodeLLM.

Dunque, è possibile concludere che la strategia self-refinement per gestire il problema della divergenza tra gli agenti programmatori nel dibattito produce, in termini di correttezza, un codice sorgente migliore rispetto a quello ottenuto applicando le altre tecniche di gestione della divergenza.

In base alle considerazioni appena fatte, si può affermare che il codice sorgente generato da Synaptic Coder è qualitativamente migliore rispetto a quello generato da un singolo Code-LLM di grandi dimensioni.

Nonostante i risultati qualitativi del sistema LLM multi-agente siano soddisfacenti, il suo costo computazionale è significativamente più grande rispetto a quello del

singolo CodeLLM, principalmente a causa della natura collaborativa, iterativa e parallela del sistema.

Analizzando i tempi di esecuzione, infatti, si è visto che il costo cresce linearmente (o più) con il numero di agenti ed il numero massimo di turni di dibattito.

Conclusioni

5.1 Impatto

In questa tesi è stato sviluppato un sistema LLM multi-agente, Synaptic Coder, in cui uno o più agenti autonomi collaborano, tramite dibattito, per assolvere compiti legati alla generazione di codice sorgente a partire da descrizioni in linguaggio naturale fornite dall'utente.

L'analisi dei risultati raggiunti da Synaptic Coder ha dimostrato che il codice sorgente generato soddisfa qualitativamente i criteri di affidabilità, manutenibilità e correttezza, ma non pienamente quello di sicurezza.

Inoltre, confrontando i risultati raggiunti dal sistema con quelli di un singolo CodeLLM dotato di un numero molto grande di parametri, in termini di correttezza, si è ottenuto che il codice sorgente generato dal sistema è, in media, migliore di quello generato dal singolo CodeLLM di 19,8%. In particolare, se si adotta nel sistema la stra-

tegia self-refinement per gestire la divergenza nel dibattito, si ha un miglioramento del codice pari a 23%.

Quest'ultimo confronto è rilevante visto che, per le leggi di scala, generalmente si procede con l'aumento dei parametri del modello (come la finestra di contesto ed altri iperparametri) per avere un miglioramento dell'efficacia e dell'accuratezza del modello nel task da svolgere.

Tuttavia, si è osservato che l'aumento del numero di agenti e del numero di round di dibattito ha un impatto significativo sulle prestazioni del sistema come il tempo di esecuzione. La gestione delle risorse (memoria, tempo di elaborazione), della coordinazione e dei canali di collaborazione tra un numero crescente di agenti introduce ulteriori complessità, come il mantenimento dell'efficienza nelle interazioni tra agenti e la prevenzione dei colli di bottiglia.

5.2 Sviluppi futuri

La ricerca nei sistemi multi-agente è un ambito della Collaborative AI in continua evoluzione ed è tutt'altro che concluso. Aspetti critici come la progettazione dei prompt, la lunghezza del contesto, il numero di agenti e i tool utilizzati richiedono ulteriori affinamenti e ottimizzazioni, specialmente con l'aumentare della complessità dei problemi.

In questo contesto, lo studio di strategie per l'ottimizzazione delle prestazioni, così come l'adozione di architetture leggere e meno costose in termini computazionali, rappresenta una direzione promettente per rendere questi sistemi più efficienti, scalabili e accessibili in scenari reali.

Negli ultimi mesi, la ricerca sui sistemi LLM multi-agente per la generazione del codice sorgente ha registrato sviluppi significativi, con nuove architetture, framework e applicazioni pratiche.

Ad esempio, è stato sviluppato AutoP2C (Paper-to-Code) [7], un framework multi-

agente basato su LLMs progettato per automatizzare la generazione di repository di codice a partire da contenuti multi-modali presenti in articoli accademici. Questo approccio supera la tradizionale generazione di codice limitata ai soli descrittori testuali, affrontando la complessità di estrarre e implementare codice eseguibile da testi, equazioni e figure.

Un altro lavoro degno di nota è ResearchCodeAgent [8]. Questo sistema LLM multi-agente mira ad automatizzare la codifica delle metodologie di ricerca descritte nella letteratura di machine learning, generando codice di alta qualità e senza errori, e migliorando l'efficienza della codifica.

Questi avanzamenti evidenziano come i sistemi LLM multi-agente stiano progressivamente ridefinendo il ruolo dell'automazione nello sviluppo software. Non si tratta più solo di generare frammenti di codice, ma di orchestrare interi processi di progettazione, analisi e implementazione. La capacità di questi sistemi di comprendere strutture complesse, collaborare su compiti specializzati e iterare sulla base di feedback, li rende strumenti fondamentali per affrontare la crescente complessità dei progetti software moderni. In prospettiva, la sinergia tra agenti intelligenti e sviluppatori umani potrebbe trasformare radicalmente la concezione stessa di sviluppo, portandoci verso una programmazione sempre più assistita, efficiente e accessibile.

Bibliografia

- [1] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, “Improving factuality and reasoning in language models through multiagent debate,” 2023. [Online]. Available: <https://arxiv.org/abs/2305.14325> (Citato alle pagine 3, 13, 35 e 37)
- [2] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhunoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, “Self-refine: Iterative refinement with self-feedback,” 2023. [Online]. Available: <https://arxiv.org/abs/2303.17651> (Citato alle pagine 7 e 36)
- [3] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, “Metagpt: Meta programming for a multi-agent collaborative framework,” 2024. [Online]. Available: <https://arxiv.org/abs/2308.00352> (Citato a pagina 13)
- [4] H. Chen, W. Ji, L. Xu, and S. Zhao, “Multi-agent consensus seeking via large language models,” 2025. [Online]. Available: <https://arxiv.org/abs/2310.20151> (Citato a pagina 13)

- [5] K.-T. Tran, D. Dao, M.-D. Nguyen, Q.-V. Pham, B. O’Sullivan, and H. D. Nguyen, “Multi-agent collaboration mechanisms: A survey of llms,” 2025. [Online]. Available: <https://arxiv.org/abs/2501.06322> (Citato a pagina 17)
- [6] T. Y. Zhuo, M. C. Vu, J. Chim, H. Hu, W. Yu, R. Widyasari, I. N. B. Yusuf, H. Zhan, J. He, I. Paul, S. Brunner, C. Gong, T. Hoang, A. R. Zebaze, X. Hong, W.-D. Li, J. Kaddour, M. Xu, Z. Zhang, P. Yadav, N. Jain, A. Gu, Z. Cheng, J. Liu, Q. Liu, Z. Wang, B. Hui, N. Muennighoff, D. Lo, D. Fried, X. Du, H. de Vries, and L. V. Werra, “Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions,” 2025. [Online]. Available: <https://arxiv.org/abs/2406.15877> (Citato a pagina 41)
- [7] Z. Lin, Y. Shen, Q. Cai, H. Sun, J. Zhou, and M. Xiao, “Autop2c: An llm-based agent framework for code repository generation from multimodal content in academic papers,” 2025. [Online]. Available: <https://arxiv.org/abs/2504.20115> (Citato a pagina 57)
- [8] S. Gandhi, D. Shah, M. Patwardhan, L. Vig, and G. Shroff, “Researchcodeagent: An llm multi-agent system for automated codification of research methodologies,” 2025. [Online]. Available: <https://arxiv.org/abs/2504.20117> (Citato a pagina 58)

Ringraziamenti

Arrivata alla conclusione di questo percorso, desidero esprimere la mia più sincera gratitudine a tutte le persone che, a diverso titolo, hanno contribuito alla realizzazione di questa tesi.

Un ringraziamento speciale va al mio relatore, il prof. Fabio Palomba, per la costante disponibilità, la guida attenta e per avermi offerto l'opportunità di cimentarmi in un argomento stimolante in cui ho potuto affinare le conoscenze acquisite e conseguire nuove competenze.

Ringrazio il dott. Antonio Della Porta, per il supporto tecnico e scientifico, e i preziosi consigli che hanno accompagnato ogni fase di questo lavoro.

Ringrazio, infine, tutti i docenti che con dedizione hanno saputo alimentare in me il desiderio di conoscenza e l'entusiasmo di proseguire questo percorso con la laurea magistrale.

Questa tesi ha contribuito a piantare un albero ad Haiti tramite il progetto Treedom.

<https://www.treedom.net/it/user/sesalab/event/sesa-random-forest>